

面試題參考：「請設計一個 AI Agent 系統」

一套可套用於多數 Agent 實作的流程與重點

這份不是標準答案，而是一個「思考框架」。面試時不必每點都講，重點是展現你有「先確認需求、再分層設計、最後想到評估與落地」的全局觀。技術詞都配了比喻，方便對非技術主管也說得清楚。

一、開場心法：先問清楚，再動手

最容易加分、也最常被忽略的一步，是「不要急著畫架構」。就像醫生不會一進門就開藥，會先問診。面試官出這種開放題，往往就是在看你會不會先釐清問題。我會先確認四件事：

- **任務本質**：是單次問答、需要跨多步驟、還是要實際操作外部系統？三者複雜度差很多。
- **自主程度**：是人保留最後決定權的「副駕駛」，還是全自動「自動駕駛」？企業內部通常從副駕駛開始，比較安全。
- **成功標準**：怎麼算成功？任務完成率、答案正確率、還是省下的工時？沒有指標就無法優化。
- **限制條件**：延遲、成本、以及資料能不能上雲——製造業常要求地端部署，這會直接影響選型。

把這四點問清楚，後面的設計才有依據。光是這個動作，就能讓面試官覺得你是做過真案子的人。

二、核心元件與心智模型

一句話的心智模型：Agent 就是一個「感知 → 決策 → 行動 → 記憶」不斷循環的系統。模型讀取現況、決定下一步、呼叫工具去行動、把結果記下來，反覆直到把任務完成——這正是它跟「單次呼叫 LLM」最大的差別。底層大致由以下幾塊組成：

元件	角色（在做什麼）	常見選擇
LLM（大腦）	負責理解、推理與生成；可依任務難易用不同大小的模型	雲端 Claude / GPT；地端 Qwen、Phi、Llama

編排控制	決定流程怎麼走：分支、迴圈、重試、何時收尾	LangGraph、LangChain、Dify、n8n
工具層	讓 Agent 能查資料、呼叫 API、操作外部系統	自訂 function calling、MCP、各式 API
檢索 (RAG)	把外部知識餵給模型，讓回答有依據、減少幻覺	文件切分 + 向量檢索
記憶	記住當前對話與長期知識	短期 context；長期 pgvector / Neo4j
護欄與驗證	檢查輸出格式與正確性、擋掉危險操作	schema 驗證、規則檢查、失敗重試
觀測與紀錄	記錄每一步，方便除錯、評估與優化	logging、trace、eval 平台

實務上不一定每塊都要做滿。簡單任務可能只要「大腦 + 幾個工具」；要接知識庫才加 RAG，要記住長期關係才加圖譜。先從最小可用版本做起，別一開始就過度設計。

三、實作流程的關鍵步驟

3.1 控制流與推理模式 (Agent 怎麼「想」)

決定 Agent 的思考方式，常見幾種：

- **ReAct**：邊想邊做，每一步推理都用工具拿到的真實結果校正。最常用、最穩，也最能減少幻覺。
- **Plan-and-Execute**：先把完整步驟規劃好再依序執行，適合流程明確的任務。
- **Reflection**：讓 Agent 檢查自己的產出再改進，能提升品質，但會增加成本與時間。
- **單 Agent vs 多 Agent**：任務很複雜時可拆成「主管 + 多個專員」分工；但建議先從單一 Agent 做起，別過度工程化。

不管哪種模式，一定要加「護欄」：設最大步數上限、偵測重複動作、卡住就交回人類。這是避免 Agent 鬼打牆、自主性失控的安全帶。

3.2 工具與技能設計 (Agent 的手腳)

工具讓 Agent 能真正做事，設計重點有三：

- **描述要清楚**：工具的名稱、用途、參數要寫得像給新人的 SOP，模型才知道「什麼時候該用哪個」。
- **粒度適中**：切太細會讓步驟爆炸、切太粗難以組合，抓在「一個工具做一件完整的事」。
- **權限分級**：查詢類工具可以放手；會「寫入、刪除、發送」的高風險工具要加確認或人工審核。

3.3 記憶與知識 (短期 + 長期)

記憶分兩種時間尺度：

- **短期記憶**：當前對話的 context，任務結束就清掉，像桌面上的便條紙。
- **長期語意記憶**：用向量資料庫 (pgvector / Milvus) 存文件與案例，做「語意相似」的模糊檢索，也就是 RAG。
- **長期關係記憶**：用知識圖譜 (Neo4j) 存實體與關係，處理「誰影響誰」的多跳查詢，製造業的製程關聯特別適合。

兩者可結合成 GraphRAG：先用圖譜定位相關範圍，再用向量補語意細節，就像先看地圖、再用放大鏡。

3.4 評估與測試 (企業最在乎、最常被略過)

Agent 是非確定性系統，同樣輸入不保證同樣輸出，所以不能用傳統「輸入 A 必得 B」的測試，要改用「評估」的思維：

- **建 golden set**：準備一組代表性的測試題與期望標準，當作考古題。
- **指標分層**：任務成功率、答案忠實度 (可用 LLM-as-judge 或人工標註)、延遲、成本、穩定性。
- **回歸測試**：每次改 prompt 或換模型都重跑一遍，確保沒有退化。

- **checklist + 測試驅動**：把「感覺它答得對」變成「逐項驗得準」，這是長專案不退化的安全網。

3.5 優化、部署與維運（讓它快、省、穩、安全）

- **效能優化**：模型分級（簡單任務用小模型）、快取重複查詢、減少步數、結構化輸出、調低 temperature。
- **資料安全**：敏感場景優先用地端 / 私有模型，做好權限控管與稽核紀錄。
- **漸進上線**：先小範圍試用、人保留決定權，建立信任後再擴大，而不是一次全面推開。
- **持續迭代**：靠前面建好的評估與紀錄，能快速發現問題並修正。

四、用一個例子串起來：跨廠品質異常彙整 Agent

把上面所有元件套到一個具體、貼近製造業的場景，會講得更有畫面：

- **需求**：四個廠的日報格式與語言不同，人工彙整很花時間。目標是自動產出統一摘要、由人最後審核（副駕駛模式）；資料敏感，要求地端。
- **架構**：地端 LLM 當大腦；工具層接各廠資料庫查詢 API；RAG 從歷史異常紀錄找相似案例；記憶用向量庫存案例、用圖譜存「製程 → 異常 → 對策」的關聯。
- **控制流**：採 ReAct——讀日報 → 判斷異常類型 → 查相似歷史案例 → 產生摘要與建議 → 交人審核；並加最大步數護欄。
- **評估**：拿過去人工彙整的報告當 golden set，比對摘要的正確率與遺漏率。
- **部署**：先在一個廠試行，流程順了再推到四廠。

這個例子的好處是「低風險、高頻、規則清楚」，很適合當第一個落地專案，也呼應你前面簡報談的落地策略。

五、面試怎麼講（順序比細節更重要）

回答這題時，講述的「順序」會決定你給人的印象。建議這樣鋪：

- **第一步**：先停一下，反問需求（展現你不會貿然動手）。
- **第二步**：畫出核心元件與「感知-決策-行動-記憶」迴圈（展現全局觀）。
- **第三步**：講控制流與護欄（展現你懂風險與穩定性）。
- **第四步**：特別著墨記憶與評估這兩塊（最能打中企業痛點，也呼應這個職缺的要求）。
- **第五步**：收在安全部署與漸進落地（展現你想的是真正上線，不是做 demo）。

「設計 Agent 的難點，不在讓它動起來，而在讓它穩定、可衡量、能被信任地長期運作。」

把這句當收尾，正好把「規格化、評估、長期維運」這些你的強項一次收攏，也跟你整份簡報的主軸首尾呼應。